LocateAnything high-quality vision-language grounding

LocateAnything high-quality vision-language grounding

1. What LocateAnything is
2. Applicable devices and capability scope
3. Preparation before the deployment
    - 3.1 Check the system and the disk
    - 3.2 Install the build and download tools
4. Downloading the model
5. Building the image grounding program
    - 5.1 Getting the source of the matching version
    - 5.2 Configuring and building the CUDA version
    - 5.3 Organising the runtime files
6. Creating the single-image run script
7. Preparing the input image
8. Running the image grounding
    - 8.1 Using the default image and target
    - 8.2 Specifying the image and the recognition target
9. How to use it with video and its performance limits
10. Frequently asked questions
11. References

1. What LocateAnything is

LocateAnything is a vision-language grounding model. It takes an image and a text description, looks for the target that matches the description in the image, and returns the bounding box coordinates of the target.

The difference from an ordinary image classification model is this: a classification model usually answers "what is in the picture", while LocateAnything also tells you where the object is in the picture. For example, if you feed in a street scene and query `person`, the model returns the position of the pedestrian; if you query `red car`, the model tries to locate the red car in the frame.

This tutorial uses the GGUF quantised model of LocateAnything-3B, run through a multimodal `llama.cpp` program adapted to that model format. After the model has been loaded, the image inference can be done offline on the Jetson itself; the first download of the model files and of the program still needs the network.

2. Applicable devices and capability scope

This tutorial is based on **Jetson Orin, JetPack 7.2.1, CUDA 13.2**. Other Jetson models or software versions have to be rechecked against the CUDA, compiler and memory compatibility, and the same parameters cannot be guaranteed to work.

The current solution is suitable for low-frequency offline grounding of a single image or of frames extracted from a video; it is not a real-time video detector. On the Orin NX 8GB, one 1280×720 image takes about 18–19 seconds in practice, and the memory usage during inference is close to the limit of the device. Do not start other large models or heavy GPU tasks at the same time as the inference.

3. Preparation before the deployment

> If you are using the factory image, the environment of locateanything is already set up, so you can skip this section up to 8.1 and run the script directly

3.1 Check the system and the disk

Open a Jetson terminal and confirm that CUDA 13.2 is available:

```
nvcc --version
free -h
df -h /home
```

`nvcc --version` should show CUDA 13.2. The model file takes about 2.8GB and building from the source needs extra disk space, so it is recommended to reserve at least 10GB on `/home`. Downloading the model and the source for the first time needs a network connection that can reach Hugging Face and GitHub.

When you want to watch the real-time memory of the device, you can run `tegrastats` and end the monitoring with `Ctrl+C`:

```
tegrastats --interval 1000
```

3.2 Install the build and download tools

```
sudo apt update
sudo apt install -y git cmake build-essential wget libssl-dev libcurl4-openssl-dev
```

Confirm that Python OpenCV is available; the scripts below use it to read and draw the images:

```
python3 -c 'import cv2; print(cv2.__version__)'
```

If it reports that `cv2` cannot be found, you can install the system package:

```
sudo apt install -y python3-opencv
```

4. Downloading the model

The main model and the vision projector are a matched pair of files. The combination specified by this tutorial has been tested on the Orin NX 8GB; download these two files and do not replace only one of them.

```
mkdir -p ~/locateanything/LocateAnything-3B-4bit
cd ~/locateanything/LocateAnything-3B-4bit

wget -c -O LocateAnything-3B-Q4_K_M.gguf \
  "https://hf-mirror.com/liodon-ai/LocateAnything-3B-imatrix-
GGUF/resolve/main/LocateAnything-3B-Q4_K_M.gguf"

wget -c -O mmproj-locateanything-3b-bf16.gguf \
  "https://hf-mirror.com/sabafallah/LocateAnything-3B-GGUF/resolve/main/mmproj-
locateanything-3b-bf16.gguf"
```

`-c` means that the download can be resumed after an interruption. Check the files once the download is complete:

```
ls -lh ~/locateanything/LocateAnything-3B-4bit/*.gguf
```

You should see a main model of about 2.0GB and a vision projector of about 833MB. The actual file sizes displayed vary slightly because of the unit conversion.

5. Building the image grounding program

5.1 Getting the source of the matching version

The vision projector type of LocateAnything needs the multimodal support of the matching version. Use the `mtmd-grounders` branch:

```
cd ~/locateanything
git clone --depth 1 --branch mtmd-grounders \
  https://github.com/yuuko-eth/llama.cpp.git
```

5.2 Configuring and building the CUDA version

The Orin NX uses the compute capability `87`. Specify the CUDA 13.2 that goes with JetPack 7.2.1 and build only the `llama-mtmd-cli` target that this tutorial needs:

```
export PATH=/usr/local/cuda-13.2/bin:$PATH
export CUDACXX=/usr/local/cuda-13.2/bin/nvcc

cd ~/locateanything/llama.cpp
cmake -S . -B build \
  -DGGML_CUDA=ON \
  -DBUILD_SHARED_LIBS=ON \
  -DCMAKE_CUDA_ARCHITECTURES=87 \
  -DCMAKE_BUILD_TYPE=Release \
  -DCUDAToolkit_ROOT=/usr/local/cuda-13.2

cmake --build build --target llama-mtmd-cli -j2
```

When the build succeeds, the following is shown at the end:

```
[100%] Built target llama-mtmd-cli
```

The CUDA build time varies with the device load and the state of the system. Do not run other large model tasks while the build is running.

5.3 Organising the runtime files

Put the program and its shared libraries into the main LocateAnything directory; the scripts below load them from there:

```
cd ~/locateanything
mkdir -p lib
cp llama.cpp/build/bin/llama-mtmd-cli ./llama-mtmd-cli
cp -a llama.cpp/build/bin/lib*.so* ./lib/
chmod +x ./llama-mtmd-cli
```

Check that the dynamic libraries are complete:

```
LD_LIBRARY_PATH="$HOME/locateanything/lib:/usr/local/cuda-13.2/lib64" \
  ldd ~/locateanything/llama-mtmd-cli | grep "not found"
```

If there is no output at all, no missing dynamic library has been found. If `not found` is shown, check whether the shared library copy above has been completed and confirm that the CUDA 13.2 path exists.

6. Creating the single-image run script

The script reads `~/locateanything/input.jpg` by default, queries `person` and saves the result with the red boxes as `out.jpg`. Copy the whole command block below to create the script:

```
cat > ~/locateanything/run_locateanything.sh <<'SCRIPT'
#!/usr/bin/env bash
set -u

BASE_DIR="$HOME/locateanything"
BIN="$BASE_DIR/llama-mtmd-cli"
MODEL="$BASE_DIR/LocateAnything-3B-4bit/LocateAnything-3B-Q4_K_M.gguf"
MMPROJ="$BASE_DIR/LocateAnything-3B-4bit/mmproj-locateanything-3b-bf16.gguf"
INPUT="${1:-$BASE_DIR/input.jpg}"
QUERY="${2:-person}"
OUTPUT="${3:-$BASE_DIR/out.jpg}"
RAW="$BASE_DIR/locate-result.txt"

if [[ ! -f "$INPUT" ]]; then
    echo "Input image does not exist: $INPUT" >&2
    exit 2
fi

export LD_LIBRARY_PATH="$BASE_DIR/lib:/usr/local/cuda-13.2/lib64:${LD_LIBRARY_PATH:-}"
echo "Recognising: $INPUT"
echo "Target: $QUERY"
```

```
"$BIN" \
    -m "$MODEL" \
    --mmproj "$MMPROJ" \
    --image "$INPUT" \
    -p "$QUERY" \
    -ngl 10 \
    -n 128 \
    --temp 0 2>&1 | tee "$RAW"
STATUS=${PIPESTATUS[0]}
if [[ $STATUS -ne 0 ]]; then
    echo "Model inference failed, exit code: $STATUS" >&2
    exit "$STATUS"
fi

python3 - "$INPUT" "$OUTPUT" "$QUERY" "$RAW" <<'PY'
import re
import sys

import cv2

input_path, output_path, query, raw_path = sys.argv[1:]
image = cv2.imread(input_path)
if image is None:
    raise SystemExit(f"Unable to read the image: {input_path}")

height, width = image.shape[:2]
with open(raw_path, "r", encoding="utf-8", errors="replace") as file:
    text = file.read()

boxes = [tuple(map(int, match)) for match in re.findall(
    r"<box>\s*<(-?\d+)>\s*<(-?\d+)>\s*<(-?\d+)>\s*<(-?\d+)>\s*</box>",
    text,
)]

for index, (x1, y1, x2, y2) in enumerate(boxes, 1):
    left = max(0, min(width - 1, round(x1 * width / 1000)))
    top = max(0, min(height - 1, round(y1 * height / 1000)))
    right = max(left + 1, min(width - 1, round(x2 * width / 1000)))
    bottom = max(top + 1, min(height - 1, round(y2 * height / 1000)))
    cv2.rectangle(image, (left, top), (right, bottom), (0, 0, 255), 3)
    cv2.putText(image, f"{query} {index}", (left, max(28, top - 8)),
                cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 0, 255), 2, cv2.LINE_AA)

if not cv2.imwrite(output_path, image):
    raise SystemExit(f"Unable to write the output image: {output_path}")
print(f"Written: {output_path}")
print(f"Parsed {len(boxes)} boxes")
PY
SCRIPT

chmod +x ~/locateanything/run_locateanything.sh
```

7. Preparing the input image

Copy the image you want to recognise to the Jetson and save it with the default file name:

```
cp /path/to/the/image.jpg ~/locateanything/input.jpg
```

You can also extract a still frame from a video as the input. The example below reads the frame at the 5th second of the video and saves it as `input.jpg`:

```
python3 - <<'PY'
import os
import cv2

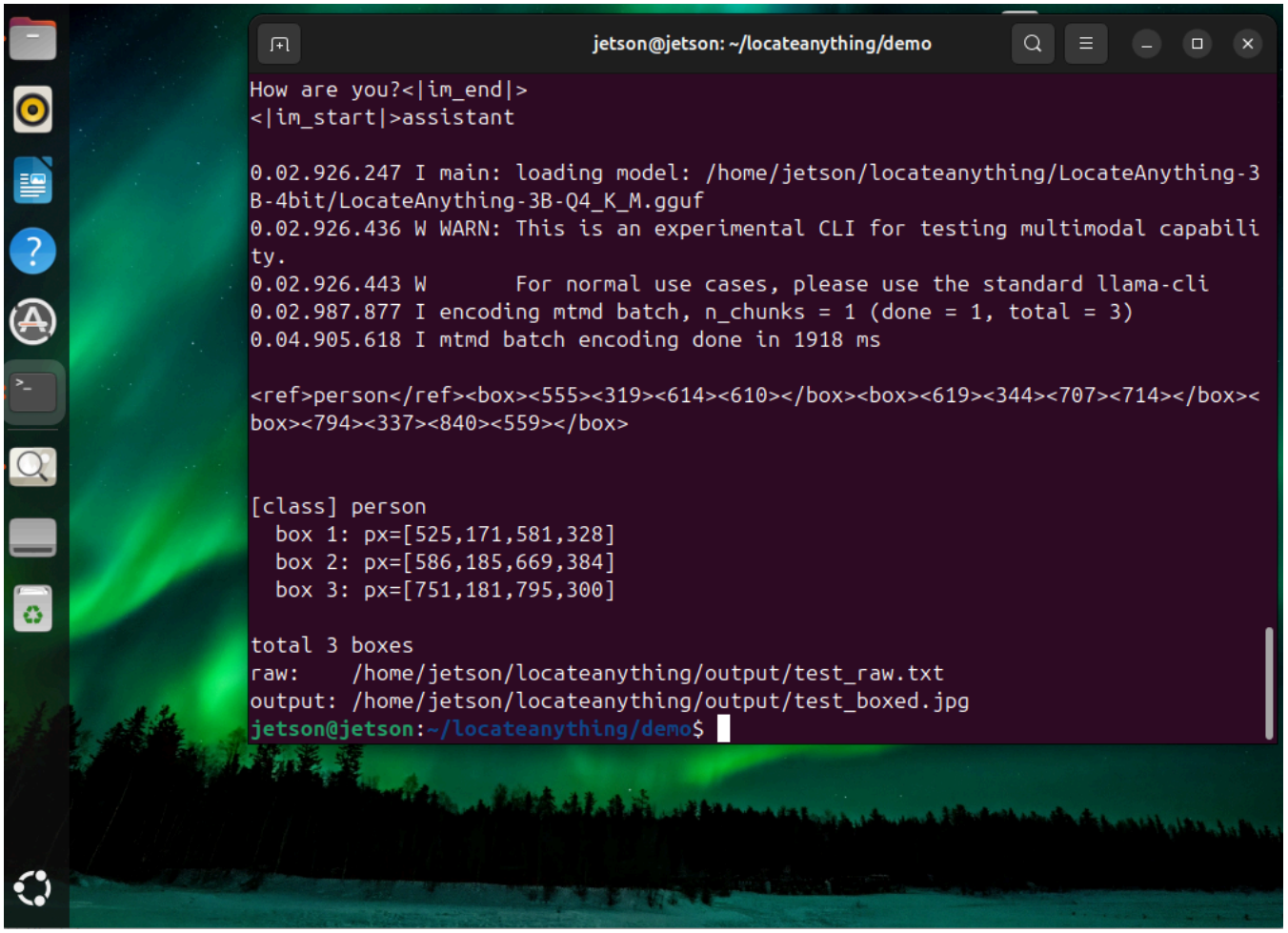
video = os.path.expanduser("~/locateanything/sample_720p.mp4")
output = os.path.expanduser("~/locateanything/input.jpg")
capture = cv2.VideoCapture(video)
capture.set(cv2.CAP_PROP_POS_MSEC, 5000)
ok, frame = capture.read()
capture.release()
if not ok:
    raise SystemExit(f"Unable to read the video frame: {video}")
if not cv2.imwrite(output, frame):
    raise SystemExit(f"Unable to save the image: {output}")
print(f"Saved: {output}")
PY
```

If the path of the video is different, change the path in the command to the actual location. This step only extracts a single image; it does not process the whole video.

8. Running the image grounding

8.1 Using the default image and target

```
cd ~/locateanything/demo
./locate_visual.sh /home/jetson/locateanything/input/test.png  "person"
```



The default parameters are:

Item	Default value
Input image	`~/locateanything/input/test.png`
Query target	`person`
Output image	`~/locateanything/output/test_boxed.jpg`
Raw model result	`~/locateanything/output/test_raw.txt`

When the recognition is finished, the terminal shows the number of detected boxes; open `out.jpg` to see the image and the red grounding boxes. Running it again overwrites `out.jpg` and `locate-result.txt`.

8.2 Specifying the image and the recognition target

Command format:

```
./run_locateanything.sh [image path] [recognition target] [output image path]
```

For example, to recognise a car in the image:

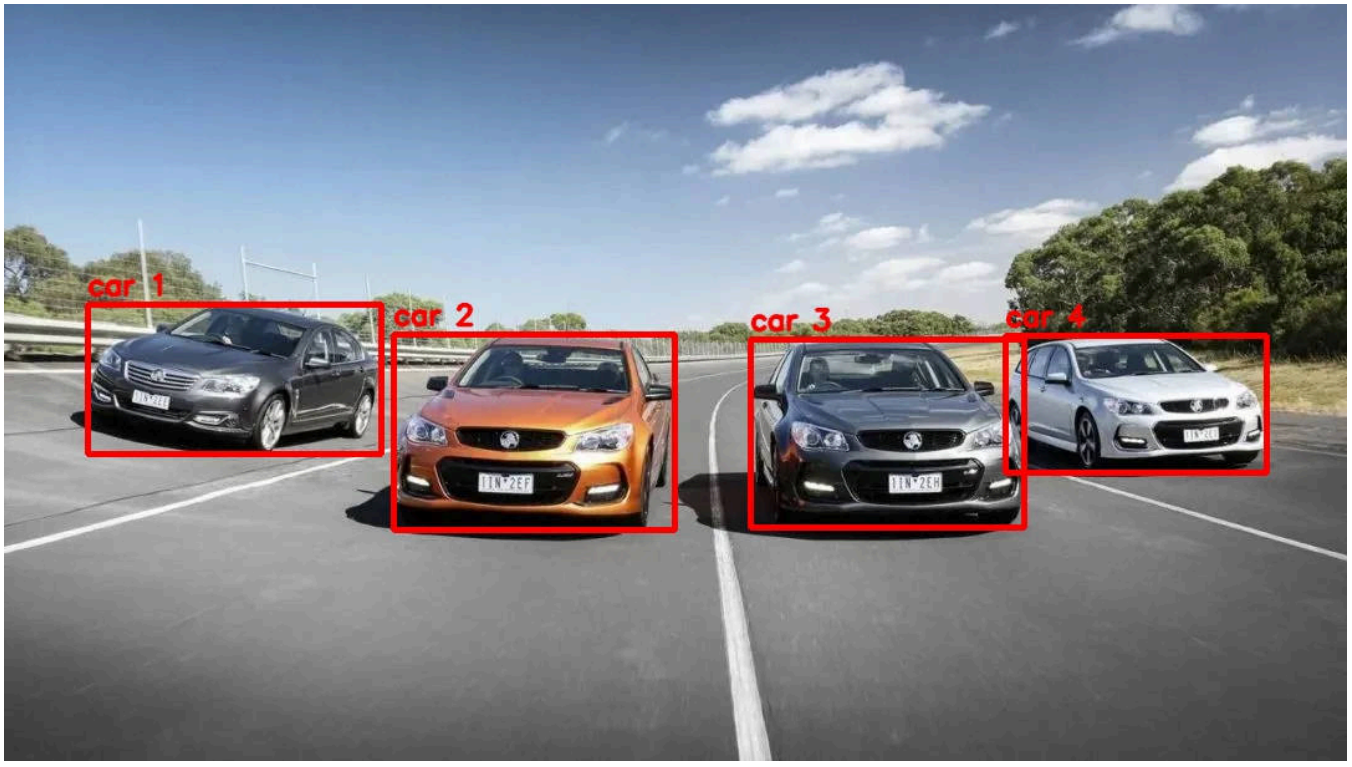
```
./run_locateanything.sh test.jpg car out.jpg
```

```
jetson@jetson:~$ cd locateanything/
jetson@jetson:~/locateanything$ ./run_locateanything.sh test.jpg car out.jpg
正在识别: test.jpg
目标: car
0.01.190.573 W load: control-looking token: 128247 '</s>' was not control-type; this is probably a bug in the model. its type will be overridden
0.04.551.972 I cmn          init: llama threadpool init, n_threads = 6
0.08.922.860 I mtmd_cli_context: chat template example:
<|im_start|>system
You are a helpful assistant<|im_end|>
<|im_start|>user
Hello<|im_end|>
<|im_start|>assistant
Hi there<|im_end|>
<|im_start|>user
How are you?<|im_end|>
<|im_start|>assistant

0.08.922.875 I main: loading model: /home/jetson/locateanything/LocateAnything-3B-4bit/LocateAnything-3B-Q4_K_M.gguf
0.08.923.108 W WARN: This is an experimental CLI for testing multimodal capability.
0.08.923.110 W       For normal use cases, please use the standard llama-cli
0.09.268.904 I encoding mtmd batch, n_chunks = 1 (done = 1, total = 3)
0.12.731.175 I mtmd batch encoding done in 3463 ms

<ref>car</ref><box><62><396><281><594></box><box><290><436><499><694></box><box><556><443><759><691></box><box><745><438><940><618></box>


已输出: out.jpg
解析到 4 个框
jetson@jetson:~/locateanything$
```



The recognition target can be an English word or a phrase; put it in quotes when it contains spaces:

```
./run_locateanything.sh test.jpg "white car" out.jpg
```

```
jetson@jetson:~/locateanything$ ./run_locateanything.sh test.jpg "white car" out.jpg
正在识别: test.jpg
目标: white car
0.01.167.335 W load: control-looking token: 128247 '</s>' was not control-type; this is probably a bug in the model. its type will be overridden
0.04.185.945 I cmn          init: llama threadpool init, n_threads = 6
0.06.622.387 I mtmd_cli_context: chat template example:
<|im_start|>system
You are a helpful assistant<|im_end|>
<|im_start|>user
Hello<|im_end|>
<|im_start|>assistant
Hi there<|im_end|>
<|im_start|>user
How are you?<|im_end|>
<|im_start|>assistant

0.06.622.402 I main: loading model: /home/jetson/locateanything/LocateAnything-3B-4bit/LocateAnything-3B-Q4_K_M.gguf
0.06.622.611 W WARN: This is an experimental CLI for testing multimodal capability.
0.06.622.616 W       For normal use cases, please use the standard llama-cli
0.06.855.632 I encoding mtmd batch, n_chunks = 1 (done = 1, total = 3)
0.10.369.776 I mtmd batch encoding done in 3515 ms

<ref>white car</ref><box><745><438><940><618></box>


已输出: out.jpg
解析到 1 个框
jetson@jetson:~/locateanything$
```



You can also specify other input and output paths:

```
./run_locateanything.sh /home/jetson/locateanything/input/test.png person
/home/jetson/locateanything/output/test_boxed.png
```

A short English target description is currently recommended. The boxes predicted by the model may miss objects or be inaccurate in position, especially for small distant objects and for adjacent objects, so check the output image manually.

9. How to use it with video and its performance limits

The current run script takes a single image and does not accept a whole video directly. To analyse a video, extract the images at the time points you need and then call the script for each of them. Every image needs a separate model inference.

On the Orin NX 8GB, one 1280×720 image takes about 18–19 seconds in practice. Processing a 40-second, 30 FPS video frame by frame needs 1200 inferences in total, which is about 6–6.3 hours; the actual time varies with the content and the system load. This solution is therefore not suitable for real-time 30 FPS video analysis.

The peak memory during the inference is about 7.0GB/7.5GB. Avoid running other large models or GPU-intensive tasks at the same time; if you run into insufficient CUDA memory, close the other memory-hungry applications and try again. Increasing the number of GPU layers is not the default solution.

10. Frequently asked questions

Symptom	How to check
`Input image does not exist`	Check the image path; the default input should be `~/locateanything/input.jpg`.
`libllama-common.so.0: cannot open shared object file`	Make sure the shared libraries are in `~/locateanything/lib/` and that you start it through the run script; do not copy only `llama-mtmd-cli` and leave out `lib/`.
`CUDA error: out of memory` or not enough system memory	Close the other large models/GPU tasks before running it; the inference memory headroom of the Orin NX 8GB is limited.
No boxes in the output, or the boxes do not match the object	Check that the query word is clear and that the object is visible in the frame, and look at the original image manually; LocateAnything is a prediction model and the result is not guaranteed to be accurate every time.
The download was interrupted	Run the `wget` command with `-c` again in the original download directory to continue the download.
The CUDA compiler was not found	Confirm the JetPack/CUDA version and set `PATH` and `CUDACXX` to the CUDA 13.2 path before building.

11. References

- Research introduction of LocateAnything: NVIDIA LocateAnything
- Model sources: LocateAnything-3B Q4_K_M and LocateAnything-3B mmproj
- Inference program source: yuuko-eth/llama.cpp, using the `mtmd-grounders` branch